

Processamento de Alto Desempenho (PAD)

Relatório de Trabalho 1 – Distância entre Vetores
Implementação e análise experimental de desempenho

Xavier Paulino Sebastião

Agosto de 2026

Resumo

Este trabalho avalia experimentalmente o cálculo da distância quadrática entre um vetor de referência e um conjunto de vetores reais. A implementação foi desenvolvida em C++ e aferida para oito tamanhos de vetor, de $T = 32$ a $T = 4096$, mantendo $N = 16384$ constante. Foram analisados tempo de execução, vetores por segundo, elementos por segundo, custo por elemento e variabilidade. Os resultados mostraram que, o tempo mediano aumentou de aproximadamente 0,446 ms para 69,448 ms. No mesmo intervalo, a taxa mediana de processamento de elementos reduziu de aproximadamente 1,176 para 0,966 bilhão de elementos por segundo, enquanto o custo por elemento aumentou de 0,851 para 1,035 ns. Além disso, os resultados revelaram crescimento aproximadamente proporcional ao volume de trabalho para os maiores valores de T , mas também uma mudança de comportamento entre $T = 64$ e $T = 256$.

1 Introdução

O problema consiste em calcular, para cada vetor x_i de um conjunto X com N vetores, a distância quadrática em relação a um vetor de referência q . Cada vetor contém T valores reais. Assim, a distância quadrática calcula-se por

$$D(q, x_i) = \sum_{j=0}^{T-1} (q_j - x_{ij})^2, \quad i = 0, \dots, N - 1. \quad (1)$$

Cada resultado é armazenado após o processamento do respectivo vetor. Com N mantido fixo e T varia, o número de elementos processados em cada execução torna-se $N \times T$ e o trabalho aritmético é esperado que cresça linearmente com T .

O objetivo experimental é observar como o aumento de T afeta o tempo de execução, as taxas de processamento e a variabilidade das aferições. Além de apresentar os tempos obtidos, busca-se verificar se o custo cresce proporcionalmente ao trabalho e identificar mudanças de comportamento ao longo da faixa analisada, que possam exigir uma explicação arquitetural mais cuidadosa. Em avaliações de desempenho, múltiplas observações e a caracterização explícita da variabilidade são importantes para evitar conclusões baseadas em execuções isoladas ou em condições transitórias da plataforma [2, 3].

2 Implementação

A implementação foi desenvolvida em C++ [7]. O vetor q , a matriz X e o vetor de resultados usam valores do tipo `double`. Durante a execução, os vetores são armazenados em memória contígua, para permitir percorrer os elementos de cada x_i sequencialmente. Para cada vetor, um acumulador recebe a soma dos quadrados das diferenças e o valor final é gravado no vetor de

resultados. Conceitualmente, o núcleo computacional avaliado corresponde ao seguinte trecho em C++:

```
for (size_t i = 0; i < N; ++i) {
    double sum = 0.0;
    for (size_t j = 0; j < T; ++j) {
        const double d = q[j] - X[i * T + j];
        sum += d * d;
    }
    distances[i] = sum;
}
```

A compilação foi realizada com GCC 10.5.0, usando C++20, `-O3` e `-march=native`, com geração adicional de um relatório de vetorização pelo compilador. Antes da execução experimental, a implementação foi verificada por um teste interno, concluído com `SELF_TEST_OK`. O uso de `-march=native` foi adotado para tornar o binário dependente das capacidades da máquina de compilação; por isso, a versão do compilador e o processador fazem parte da descrição experimental. O ambiente Python utilizado nos scripts de calibração, análise e geração de gráficos foi executado em Conda, no ambiente `t0_vector_distance`.

3 Metodologia experimental

3.1 Plataforma e ambiente

As aferições foram executadas em um notebook Lenovo ThinkPad T530 equipado com processador Intel Core i5-3320M a 2,60 GHz, arquitetura x86-64, família 6, modelo 58. A CPU disponibiliza AVX, SSE4.1 e SSE4.2, entre outras extensões. A execução foi fixada no CPU lógico 2, selecionado deterministicamente. O sistema operacional utilizado foi GNU/Linux Ubuntu 24.04.4 LTS, com GCC 10.5.0 como compilador C++.

O ambiente Conda utilizado no desenvolvimento foi denominado `t0_vector_distance`, mas o benchmark C++ foi compilado com `/usr/bin/g++`, evitando o compilador fornecido pelo Conda. O compilador do sistema foi adotado porque o binário produzido pelo compilador Conda exigia nível ISA superior ao suportado pelo processador experimental.

3.2 Escolha de N

O valor de N foi determinado antes dos experimentos por um procedimento de calibração e permaneceu inalterado em todos os experimentos. A calibração selecionou $N = 16384$. O critério foi escolher a maior potência de dois compatível com a capacidade de memória definida para a maior entrada. Durante a calibração, a memória disponível era aproximadamente 2,44 GiB; foi reservado no máximo 25% desse valor para a alocação principal. Para $T = 4096$, a alocação estimada foi de aproximadamente 512,2 MiB, abaixo da capacidade de aproximadamente 624,7 MiB. O alvo inicial de 10 ms para $T = 32$ não foi atingido antes do limite imposto pela memória; por isso, $N = 16384$ foi mantido como o maior valor admissível pelo critério definido previamente.

A calibração também verificou a adequação do temporizador. Para $T = 32$, obteve-se mediana de 0,530.868 ms na etapa de calibração, enquanto o intervalo mediano entre leituras consecutivas de relógio `CLOCK_MONOTONIC_RAW` [1] foi de 32 ns, equivalente a aproximadamente 0,006028% do tempo do kernel. Assim, o custo nominal da leitura do relógio foi considerado desprezível frente ao menor tempo aferido.

3.3 Dados e região cronometrada

Os dados são gerados pelo próprio programa com semente fixa 42, para permitir repetir a inicialização. Antes de iniciar a coleta de tempo são concluídas a alocação de q , de X e do vetor de

resultados, bem como a geração e inicialização dos valores. Nenhuma dessas operações é repetida dentro da região aferida.

A medição começa imediatamente antes do cálculo de $D(q, x_0)$ e termina após o cálculo e armazenamento de $D(q, x_{N-1})$. Portanto, geração de dados, alocação, entrada ou saída e apresentação de resultados não integram o tempo reportado. Foram realizadas 5 execuções de aquecimento antes das medições. Os experimentos tiveram 30 aferições por tamanho, totalizando 240 observações, organizadas em cinco blocos de seis repetições. A realização de múltiplas aferições permite caracterizar a variabilidade dos tempos de execução, aspecto importante em avaliações experimentais de desempenho [8]. A ordem dos tamanhos foi intercalada entre blocos para reduzir a associação sistemática entre um valor específico de T e o momento em que ele é executado. Esse cuidado é relevante porque fatores aparentemente secundários da configuração e da ordem experimental podem alterar resultados de desempenho [6].

Foram avaliados os tamanhos exigidos: $T \in \{32, 64, 128, 256, 512, 1024, 2048, 4096\}$.

3.4 Índices de desempenho

Além do tempo de execução t , foram calculadas as seguintes taxas exigidas no trabalho:

$$P_v = \frac{N}{t}, \quad P_e = \frac{NT}{t}, \quad (2)$$

onde P_v representa vetores por segundo e P_e elementos por segundo. Como índice complementar, foi utilizado o custo temporal por elemento,

$$C_e = \frac{t \times 10^9}{NT} \quad \text{ns/elemento.} \quad (3)$$

Também foi calculada a razão do tempo mediano em relação a $T = 32$ e comparada à razão de trabalho $T/32$. Para caracterizar a dispersão, foram considerados quartis, boxplots e coeficiente de variação (CV). A mediana foi adotada como medida central principal por ser menos sensível a observações ocasionalmente elevadas.

3.5 Reprodutibilidade

Todos os arquivos dos experimentos são guardados no repositório público, que reúne o código-fonte, o **Makefile**, os scripts de calibração, execução, análise e geração dos gráficos, os dados brutos e derivados, o relatório de vetorização e o relatório em PDF. Os experimentos são executados com a seguinte sequência de comandos:

```
conda activate t0_vector_distance
export CXX=/usr/bin/g++
make clean
make release
make test
make vectorization
python scripts/calibrate_n.py
cat config/experiment.conf
./scripts/run_experiments.sh
python scripts/analyze_results.py
python scripts/plot_results.py
```

Os arquivos de configuração e as medições da plataforma experimental integram o repositório. Os valores $N = 16384$ e CPU lógico 2 correspondem à configuração empregada nestes experimentos.

4 Resultados

4.1 Resumo quantitativo

A Tabela 1 resume os principais resultados. No geral, observou-se que o tempo mediano cresce continuamente com T . Como N é constante, a taxa de vetores por segundo reduziu muito, enquanto a taxa de elementos por segundo apresentou redução mais moderada e tende a se estabilizar nos maiores tamanhos.

Tabela 1: Resumo dos resultados por tamanho de vetor.

T	Tempo mediano (ms)	P_v (10^6 vet./s)	P_e (10^9 elem./s)	ns/elem.
32	0,446	36,74	1,176	0,851
64	0,900	18,20	1,165	0,858
128	1,999	8,20	1,049	0,953
256	4,254	3,85	0,986	1,014
512	8,583	1,91	0,977	1,023
1024	17,216	0,952	0,974	1,026
2048	34,438	0,476	0,974	1,026
4096	69,448	0,236	0,966	1,035

4.2 Tempo de execução e crescimento do trabalho

A Figura 1 mostra o tempo mediano. Entre $T = 32$ e $T = 4096$, o tamanho do vetor cresceu 128 vezes, enquanto o tempo mediano passou de aproximadamente 0,446 ms para 69,448 ms, razão de aproximadamente 155,7. Portanto, o crescimento observado foi maior que o aumento nominal do número de elementos.

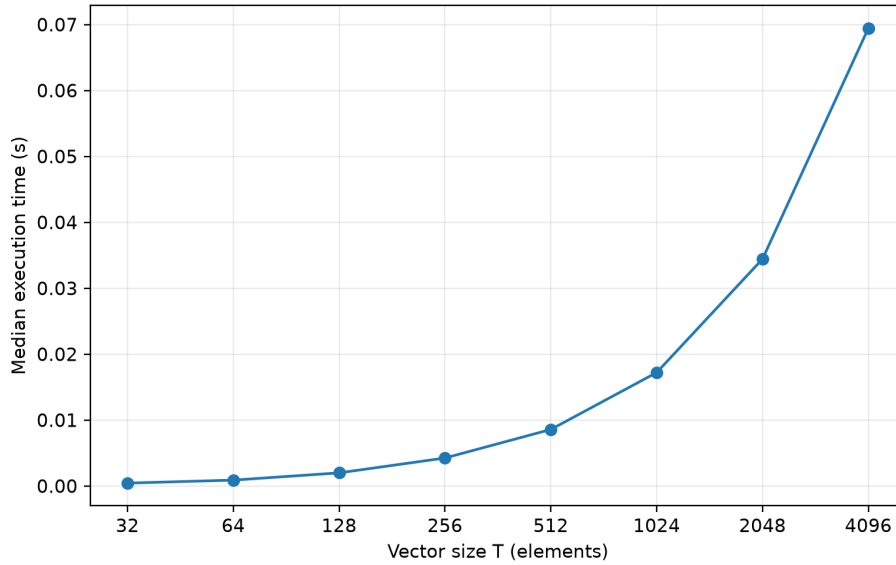


Figura 1: Tempo mediano de execução em função de T .

A comparação normalizada da Figura 2 tornou essa diferença mais visível. Em $T = 64$, o tempo acompanhou praticamente a duplicação do trabalho. A partir de $T = 128$, a curva observada permaneceu acima da referência $T/32$. Para $T = 4096$, o tempo foi cerca de 155,7 vezes o caso-base, contra uma razão de trabalho de 128. Isso equivale a um custo por unidade de trabalho aproximadamente 21,7% maior que no caso $T = 32$.

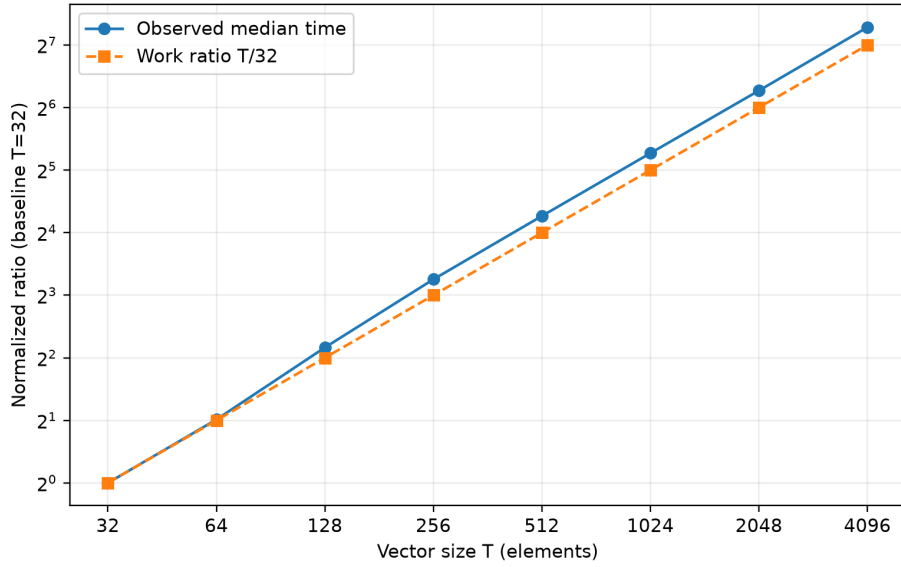


Figura 2: Tempo mediano normalizado por $T = 32$ e razão nominal de trabalho $T/32$.

Notou-se que esse resultado não significou que o algoritmo tenha deixado de apresentar comportamento linear em T . Para os tamanhos maiores, dobrar T produziu aproximadamente o dobro do tempo: de $T = 1024$ para $T = 2048$, por exemplo, a mediana passou de 17,216 ms para 34,438 ms. O que mudou foi a constante efetiva de custo por elemento entre os menores tamanhos e o regime observado a partir de aproximadamente $T = 256$.

4.3 Taxas de processamento

A taxa de vetores processados por segundo, apresentada na Figura 3, diminuiu de aproximadamente 36,74 milhões de vetores/s em $T = 32$ para 0,236 milhão de vetores/s em $T = 4096$. Essa queda é esperada, visto que cada vetor no maior caso contém 128 vezes mais elementos que no menor caso. Assim, pode-se entender que P_v caracteriza a capacidade de concluir vetores, mas isoladamente não mede a eficiência por elemento.

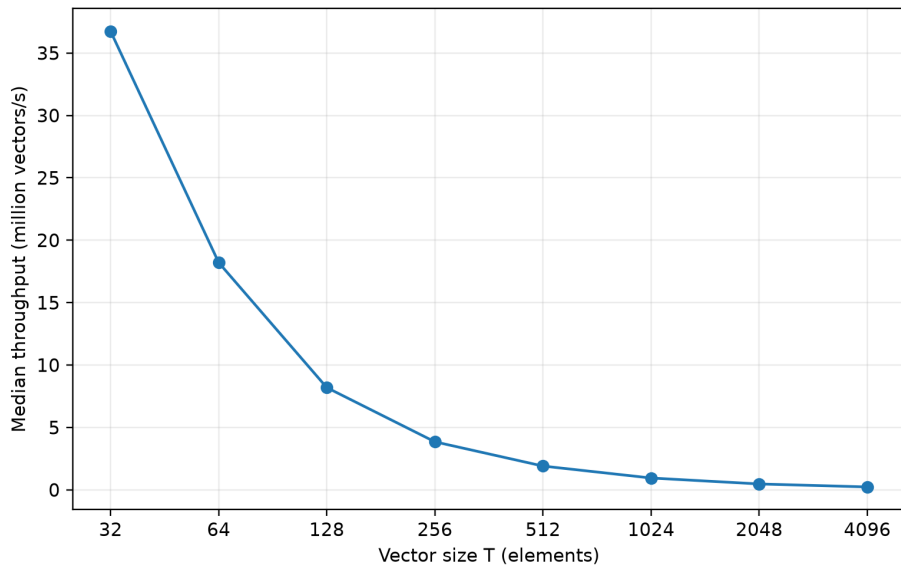


Figura 3: Taxa mediana de processamento de vetores.

A taxa de elementos, na Figura 4, foi estabelecida para comparar o custo por unidade de

trabalho. Ela permaneceu próxima de 1,17 bilhão de elementos/s para $T = 32$ e $T = 64$, reduziu muito entre $T = 64$ e $T = 256$ e, depois, aproximou-se de um patamar entre 0,97 e 0,99 bilhão de elementos/s. Em $T = 4096$, a mediana foi aproximadamente 0,966 bilhão de elementos/s.

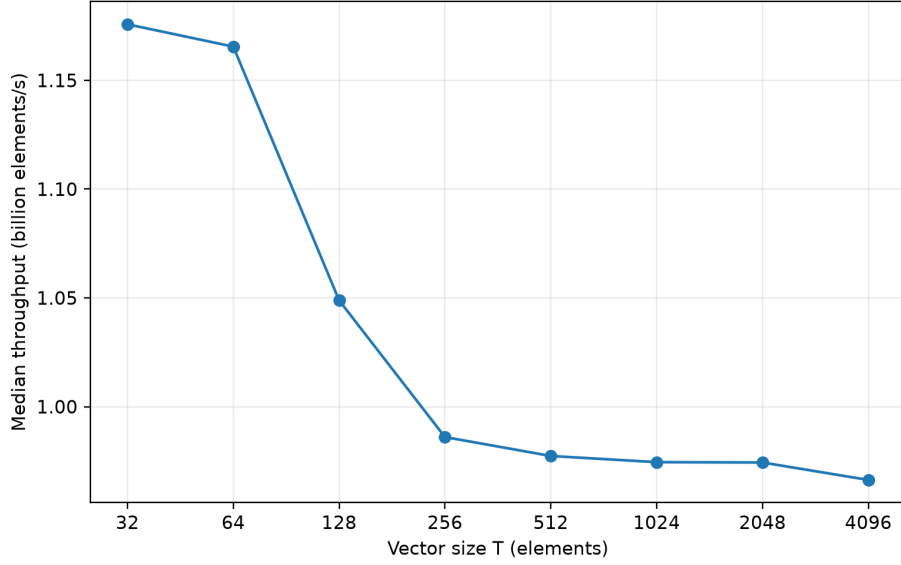


Figura 4: Taxa mediana de processamento de elementos.

4.4 Custo por elemento e mudança de comportamento

A Figura 5 apresenta o inverso da taxa de elementos. O custo mediano foi 0,851 ns/elemento em $T = 32$ e 0,858 ns/elemento em $T = 64$. Em seguida aumentou para 0,953 ns/elemento em $T = 128$ e 1,014 ns/elemento em $T = 256$. De $T = 512$ a $T = 4096$, teve variação apenas de aproximadamente 1,023 a 1,035 ns/elemento.

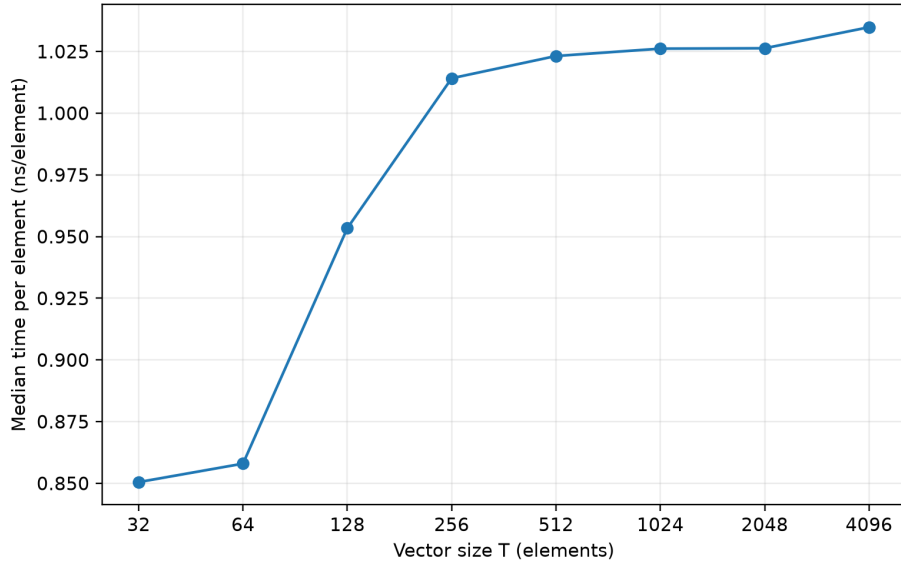


Figura 5: Custo mediano por elemento processado.

Os dados indicam uma mudança de comportamento principalmente entre $T = 64$ e $T = 256$, seguida por relativa estabilidade para os maiores tamanhos. Embora essa mudança seja observada experimentalmente, entende-se que sua causa não pode ser determinada apenas a partir dos tempos medidos. O vetor q ocupa 256 bytes em $T = 32$ e 32 KiB em $T = 4096$, enquanto a matriz X cresce

de aproximadamente 4 MiB para 512 MiB no mesmo intervalo. Esse aumento no volume de dados pode afetar a interação com a hierarquia de memória. Em processadores Intel, acesso sequencial, reutilização em cache, prefetch de hardware e tráfego de memória estão entre os fatores que podem afetar o desempenho desse tipo de percurso [4]. No entanto, os experimentos não incluíram contadores de cache, largura de banda de memória ou outros indicadores microarquiteturais que permitam estabelecer a origem da mudança observada. A infraestrutura `perf_event` do Linux oferece eventos como ciclos, instruções, referências e faltas de cache, mas esses indicadores não integram os dados analisados aqui [5]. Assim, os resultados não permitem associar essa mudança especificamente ao comportamento da cache, ao acesso à memória principal ou a outro componente da microarquitetura.

4.5 Variabilidade

A Figura 6 mostra a distribuição das 30 observações de cada tamanho. Os coeficientes de variação foram aproximadamente 14,26%, 7,10%, 7,50%, 3,71%, 1,41%, 1,13%, 1,29% e 4,13% para $T = 32$ a $T = 4096$, respectivamente. A região entre $T = 512$ e $T = 2048$ foi observada como a mais estável. Nos menores tamanhos, pequenas interferências externas podem representar fração maior de um kernel que dura menos de poucos milissegundos. Esse efeito é distinto do custo do temporizador, que correspondeu a uma fração desprezível do menor tempo aferido na calibração.

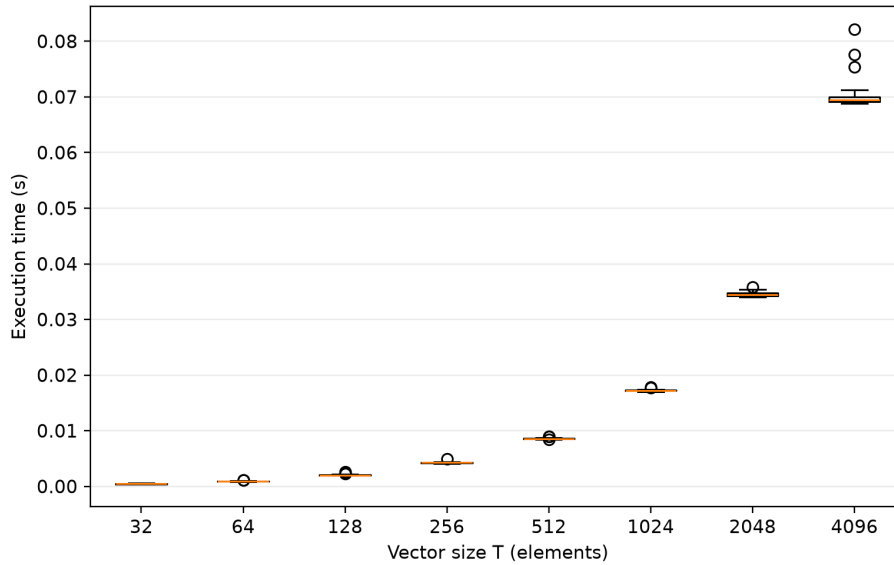


Figura 6: Distribuição dos tempos de execução nas repetições. Os círculos indicam observações além dos limites convencionais do boxplot; elas foram preservadas na base de dados.

Em $T = 4096$, aparecem algumas observações superiores afastadas do núcleo da distribuição; o máximo observado foi aproximadamente 82,167 ms, contra mediana de 69,448 ms. Esses valores não foram removidos, pois não houve evidência independente de erro de medição.

5 Discussão

Os resultados são coerentes com a estrutura do cálculo realizado, pois, mantendo N constante, a quantidade de elementos processados cresce proporcionalmente a T , e o tempo de execução apresenta crescimento aproximadamente linear, sobretudo a partir de $T = 256$. Os índices normalizados mostram, entretanto, que o custo por elemento não permanece constante em toda a faixa avaliada. Os menores vetores apresentam maior taxa de processamento de elementos e menor custo por elemento. Entre $T = 64$ e $T = 256$, observa-se uma redução dessa taxa, seguida

por relativa estabilidade para os maiores tamanhos.

A taxa de vetores por segundo, quando analisada isoladamente, não representa a eficiência por elemento. Sua redução à medida que T aumenta não indica, por si só, uma perda equivalente de eficiência, pois o trabalho associado ao processamento de cada vetor também aumenta. Para essa comparação, P_e e o tempo em ns/elemento são mais informativos. Esses dois índices, porém, não constituem evidências independentes, pois um corresponde ao inverso do outro, considerando a conversão de unidades. Eles apresentam, portanto, duas formas de expressar o mesmo comportamento observado.

Os resultados também evidenciam a importância de realizar múltiplas aferições para cada tamanho. A dispersão não é uniforme entre as configurações, e algumas observações apresentam tempos superiores aos valores centrais, sobretudo em $T = 4096$. As repetições, juntamente com a mediana, os boxplots e o coeficiente de variação (CV), permitem caracterizar tanto a tendência central quanto a variabilidade das aferições. Essa prática é coerente com recomendações de benchmarking que enfatizam repetição, análise de variabilidade e cautela na interpretação de diferenças de desempenho [2, 8, 3]. As medições realizadas, contudo, não fornecem informações sobre temperatura, frequência instantânea da CPU, interrupções do sistema operacional ou eventos de hardware associados à hierarquia de memória. Assim, esses fatores podem ser considerados na interpretação dos resultados, mas não podem ser identificados, a partir dos dados coletados, como causas das variações observadas.

6 Conclusão

A implementação calculou e armazenou as distâncias quadráticas para $N = 16384$ vetores em todos os tamanhos avaliados. O tempo mediano aumentou de aproximadamente 0,446 ms em $T = 32$ para 69,448 ms em $T = 4096$. Nesse intervalo, o número de elementos processados por execução aumentou 128 vezes, enquanto o tempo cresceu aproximadamente 155,7 vezes. Esse comportamento foi acompanhado pela redução da taxa de processamento de cerca de 1,176 para 0,966 bilhão de elementos por segundo e pelo aumento do custo mediano por elemento de 0,851 para 1,035 ns.

A principal mudança de comportamento foi observada entre $T = 64$ e $T = 256$, seguida por relativa estabilidade do custo por elemento nos maiores tamanhos. A variabilidade foi mais elevada nos menores tamanhos, diminuiu entre $T = 512$ e $T = 2048$ e voltou a crescer em $T = 4096$. As observações discrepantes foram preservadas, pois não foram identificadas evidências de erro que justificassem sua exclusão.

Os resultados caracterizam o efeito do aumento de T sobre o desempenho da implementação e mostram que o crescimento do tempo é aproximadamente linear para os maiores tamanhos, embora o custo por elemento não permaneça constante em toda a faixa avaliada. As aferições realizadas, entretanto, não permitem determinar a causa microarquitetural da mudança observada nem associá-la especificamente ao comportamento da cache, ao acesso à memória principal ou à vetorização.

Referências

- [1] G. Buttazzo and G. Lipari, “Ptask: An educational C library for programming real-time systems on Linux,” in *2013 IEEE 18th Conference on Emerging Technologies & Factory Automation (ETFA)*, pp. 1–8, 2013.
- [2] T. Hoefer and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, 2015.

- [3] C. Curtsinger and E. D. Berger, “Stabilizer: Statistically sound performance evaluation,” *ACM SIGARCH Computer Architecture News*, vol. 41, no. 1, pp. 219–228, 2013.
- [4] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Version 050, 2026. Disponível em: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel64-and-ia32-architectures-optimization.html>.
- [5] M. Kerrisk and Linux man-pages contributors, “perf_event_open(2) – set up performance monitoring,” *Linux man-pages*, 2026. Disponível em: https://man7.org/linux/man-pages/man2/perf_event_open.2.html.
- [6] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!,” *ACM SIGPLAN Notices*, vol. 44, no. 3, pp. 265–276, 2009.
- [7] H. Schildt, *C++: The Complete Reference*. McGraw-Hill, 1998.
- [8] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pp. 63–74, 2013.